Comprehensive Practice Guide for Requirements Engineering

We have provided you with a **use case** for a system called **Active Side Guard Assist (ASGA)**. This use case outlines various features and functional requirements of the system. Your task is to **analyze the system** and identify what kinds of **explainability requirements** might be important for users—i.e., what the system should explain, when, how, and to whom.

You will be working in three phases using the practices assigned to you:

◇ What You Need to Do

1. **Understand the Use Case:**
    - Read through the ASGA use case carefully.
    - Get familiar with how the system functions and interacts with users.
2. **Identify Explainability Requirements:**
    - Think about where and why the user might need explanations.
    - Focus on what the user needs to understand in order to trust or make sense of the system's behavior.
3. **Apply the Following Practices (as assigned):**

◇ *Phase 1: Requirements Elicitation*

Use your assigned technique (e.g., semi-structured interview, brainstorming, or task analysis) to gather explainability requirements from the provided scenario.

◇ *Phase 2: Requirements Specification / Modeling*

Based on the requirements identified, use your assigned method (e.g., structured natural language, use case modeling, or softgoal diagrams) to write or model the requirements clearly.

◇ *Phase 3: Requirements Validation*

Evaluate the requirements using your assigned validation technique (e.g., user feedback session, walkthrough, or compliance check) to assess whether they are clear, relevant, and complete.

☑ Please Remember:

- Use the templates provided to complete each task.
- Follow the **think-aloud protocol** by verbalizing your thoughts as you work through the tasks.
- Note down any **challenges, doubts, or observations**—we'll discuss these in the group session.

Each section below includes:

- **Overview of the Practice** (What it is and why it matters)
- **Key Concepts** (Simple explanations)
- **Step-by-step instructions** (How to perform the task)
- **Example based on "Active Side Guard Assist"**

Requirements Elicitation Practices

1. Semi-Structured Interviews (Observer: Jonas)

Overview: This is a guided conversation to explore what users want or expect from a system. It's structured around a set of core questions but allows flexibility to follow interesting points.

Key Concepts:

- **Elicitation:** The process of collecting information or requirements from people.
- **Explainability:** Users should understand how or why a system makes a decision.

How to Carry Out the Task:

1. Read the system overview of "Active Side Guard Assist."
2. For this task you need two stakeholders(two students will serve as stakeholders).
3. Ask the students/interviwees what type of explanation they need in different scenarios considering the given use-case.
4. Write the questions you have asked.
5. Listen carefully and write down answers.
6. Ask follow-up questions if needed.

Example:

- *Q:* What explanations do you expect when the system activates?
- *A:* I want to know what triggered the alert, like if a vehicle was in my blind spot.

---

2. Brainstorming Sessions (Observer: Lucas)

Overview: Brainstorming is a creative group activity where people generate as many ideas as possible about what a system should explain or do.

Key Concepts:

- **Collaboration:** Everyone shares ideas without judgment.
- **Idea Expansion:** Build on each other's ideas.

How to Carry Out the Task:

1. Gather the group and explain the goal (e.g., what the system should explain).
2. Ask them to share ideas freely.
3. Note down every idea.
4. After idea generation, discuss and prioritize the best ideas.

Example:

- Idea: Show a visual when an obstacle is detected.
- Priority: High – helps user understand system's action.

---

3. Task Analysis (Observer: Habiba)

Overview: Task analysis breaks down what users do step-by-step and helps identify where explanations are needed.

Key Concepts:

- **Tasks:** Actions the user takes.
- **Explainability Need:** When users must understand what the system is doing.

How to Carry Out the Task:

1. Identify a task involving the system (e.g., lane change).
2. Break it down into steps.
3. Think about what the user might want explained at each step.

Example:

- Task: Lane change
- Step: System detects vehicle → Explain: "Vehicle detected in side zone."

Requirements Specification Practices

4. Structured Natural Language Templates (Observer: Lucas)

Overview: Specification means writing down the requirements in a clear and consistent way.

Key Concepts:

- **Requirement:** A specific thing the system should do.
- **Structured Language:** Use of clear, repeatable format to avoid confusion.

How to Carry Out the Task:

1. Use the sentence pattern: "The system shall..."
2. Include what it does and why.
3. Write it simply and clearly.

Example:

- The system shall explain why an alert was triggered during a lane change, to build user trust.

5. Use Case Modeling (Observer: Jonas)

Overview: Use cases show how users interact with the system step by step. A use case diagram shows how a user (or another system) interacts with a system to achieve specific goals. It's a visual way to understand the **functional requirements** of a system.

Key Concepts:

- **Actor:** The user or system interacting.
- **Scenario:** A series of steps.

How to Carry Out the Task:

1. Identify the user (e.g., driver).
2. Describe what they do.
3. Show how the system responds.

Example:

✍ Steps to Create a Use Case Diagram

Step 1: Identify the System

- Clearly define the **system** you are designing (e.g., *Active Side Guard Assist*).
- This will be the box that contains all use cases.

Step 2: Identify the Actors

- Actors are **users** or **external systems** that interact with your system.
- Ask: *Who will use or interact with this system?*

  Examples:

  - Driver (Primary actor)
  - Vehicle sensors (Secondary system actor)

Step 3: Identify the Use Cases

- Use cases are the **actions** or **services** the system provides for the actors.
- Think about what the user wants the system to do.

  Examples:

  - Detect object in blind spot
  - Alert driver with warning
  - Provide explanation for alert
  - Disable assist system

Step 4: Draw the Diagram Elements

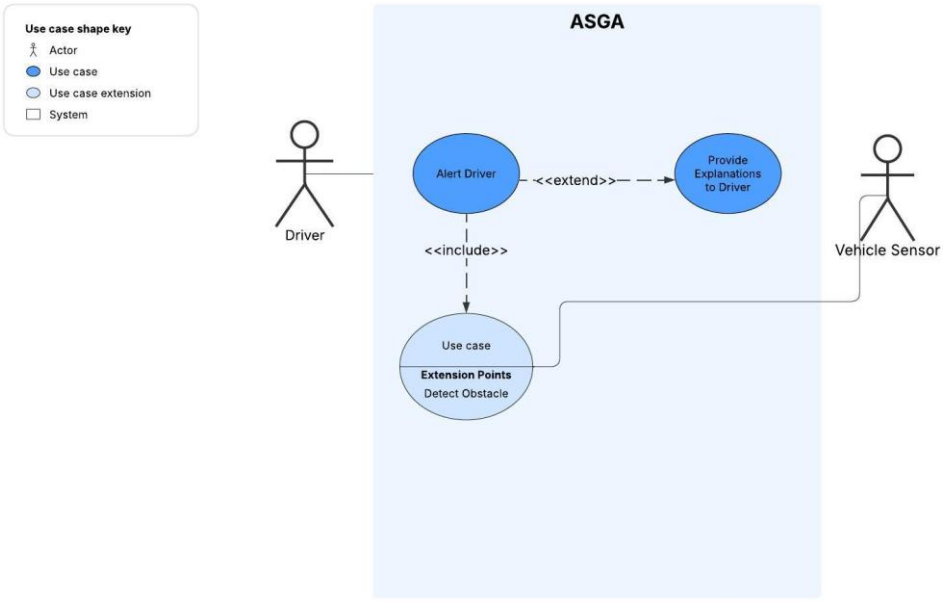
Use any tool like draw.io, Lucidchart, or even paper.

- **Draw the System Boundary (box):** Label it (e.g., *Active Side Guard Assist*).
- **Place the Actors:** Outside the box, on the left/right side.
- **Add Use Cases:** Inside the box as ovals.
- **Connect Actors to Use Cases:** Use straight lines to show interactions.

Step 5: Define Relationships (if needed)

Use different types of arrows or connections:

- <<include>> – If one use case **always includes** another
  → e.g., *Alert Driver* includes *Detect Obstacle*
- <<extend>> – If one use case **sometimes extends** another
  → e.g., *Provide Explanation* may extend *Alert Driver*



---

6. Softgoal Interdependency Graphs (SIGs) (Observer: Habiba)

Overview: SIGs help visualize goals and how they support or conflict with each other.

Key Concepts:

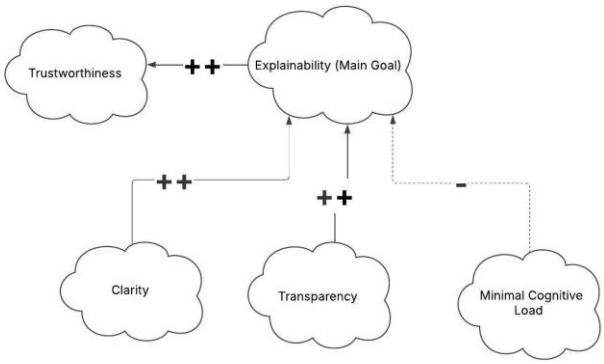
- **Softgoal:** A quality like trust, usability, or explainability.
- **+/- Impact:** Whether a feature supports or hurts a goal.

How to Carry Out the Task:

1. Write down your main goal (e.g., explainability).
2. List things that affect it positively or negatively.
3. Draw arrows to show relationships.

Example:

- Goal: Explainability
    - Clear visual alert
    - Too much text (cognitive load)



Softgoals Defined

Softgoal ID	Softgoal	Description
SG1	Explainability	The system should explain its actions clearly and meaningfully.
SG2	Clarity	Explanations should be simple and easy to understand.
SG3	Transparency	The system should make its reasoning process visible.
SG4	Minimal Cognitive Load	The explanation should not overwhelm the user.
SG5	Trustworthiness	Good explanations contribute to building trust in the system.

How to Interpret This Graph

- **SG1 (Explainability)** is the main softgoal.
- It is **positively influenced** by **clarity** (SG2) and **transparency** (SG3).
- However, **increasing detail** (to improve clarity or transparency) may result in **higher cognitive load** (SG4), which negatively impacts the main goal.
- All three subgoals (SG2–SG4) collectively impact **trustworthiness** (SG5), which is an indirect goal of explainability.

Contribution Link

- Shows how one softgoal affects another.
- Can be **positive (+)** or **negative (-)**.
- Often annotated using the following symbols:

Symbol	Meaning
++	Strong positive impact
+	Weak positive impact
−	Weak negative impact
−−	Strong negative impact

Requirements Validation Practices

7. Structured User Feedback Session (Observer: Jonas)

Overview: This is about asking users what they think of the explanations provided by the system.

Key Concepts:

- **Feedback:** User opinions.
- **Usability:** How understandable and useful the system is.

How to Carry Out the Task:

1. For this task you need two stakeholders(two students will serve as stakeholders)
2. Show the stakeholder the system explanation (specified requirements; outcome of phase 2).
3. Prepare questions to ask about the validation of requirements.
4. Ask prepared questions (e.g., Was this clear?).
5. Record their answers and suggestions.

Example:

- Q: Was the explanation clear?
- A: Yes, but it would help to see it visually too.
- Detecting missing requirements
- Unrealistic requirements
- Highlighting communication gaps if any

---

8. Walkthroughs (Observer: Lucas

Overview: A walkthrough is a step-by-step review of each requirement to check clarity and correctness.

Key Concepts:

- **Step-by-Step Review:** Going through each requirement.
- **Clarity Check:** Is it understandable?

How to Carry Out the Task:

1. Read the requirement.
2. Ask yourself: Is it clear? Is anything missing?
3. Note any issues or questions.

Example:

- Requirement: System explains blind spot detection.
- Issue: It doesn't say how the explanation is shown.

---

9. Compliance Checking (Observer: Habiba)

Overview: Compliance checking ensures the system meets legal or ethical rules (like being transparent).

Key Concepts:

- **Regulation:** Rules like GDPR.
- **Checklist:** A way to ensure everything required is included.

How to Carry Out the Task:

1. Read the requirement.
2. Check if it follows known standards.
3. Mark Yes/No and explain why.

Example:

- Requirement: System shows reason for alert.
- GDPR Compliance: Yes – meets transparency requirement.